DEEP LEARNING AND APPLICATIONS

MR20-1CS0158

UNIT II

Prepared by

Dr.M.Narayanan

Professor

Department of CSE

Malla Reddy University, Hyderabad

DEEP LEARNING AND APPLICATIONS

MR20-1CS0158

UNIT II

Tensor Flow: Computational Graph, Key highlights, Creating a Graph, Regression example, Gradient Descent, Tensor Board, Modularity, Sharing Variables, Keras Perceptrons: What is a Perceptron, XOR Gate.

Activation Functions: Sigmoid, ReLU, Hyperbolic Fns, Softmax

Text Book

1. Goodfellow, I., Bengio, Y., and Courville, A., Deep Learning, MIT Press, 2016

What is TensorFlow?

- TensorFlow is a popular framework of machine learning and deep learning. It is a free and open-source library which is released on 9 November 2015 and developed by Google Brain Team. It is entirely based on Python programming language and use for **numerical computation and data flow**, which makes machine learning faster and easier.
- TensorFlow can train and run the deep neural networks for image recognition, handwritten digit classification, recurrent neural network, word embedding, natural language processing, video detection, and many more. TensorFlow is run on multiple CPUs or GPUs (Graphics processing unit) and also mobile operating systems.
- The word TensorFlow is made by two words, i.e., Tensor and Flow
  - **Tensor is a multidimensional array**
  - **Flow is used to define the flow of data in operation.**
- **TensorFlow is used to define the flow of data in operation on a multidimensional array or Tensor**

Components of TensorFlow

Tensor

- The name TensorFlow is derived from its core framework, "Tensor." A tensor is a vector or a matrix of n-dimensional that represents all type of data. All values in a tensor hold similar data type with a known shape. The shape of the data is the dimension of the matrix or an array.
- A tensor can be generated from the input data or the result of a computation. In TensorFlow, all operations are conducted inside a graph. The group is a set of calculation that takes place successively. Each transaction is called an **op node** are connected.

Graphs

- TensorFlow makes use of a graph framework. The chart gathers and describes all the computations done during the training.

Consider the following expression a= (b+c)*(c+2)

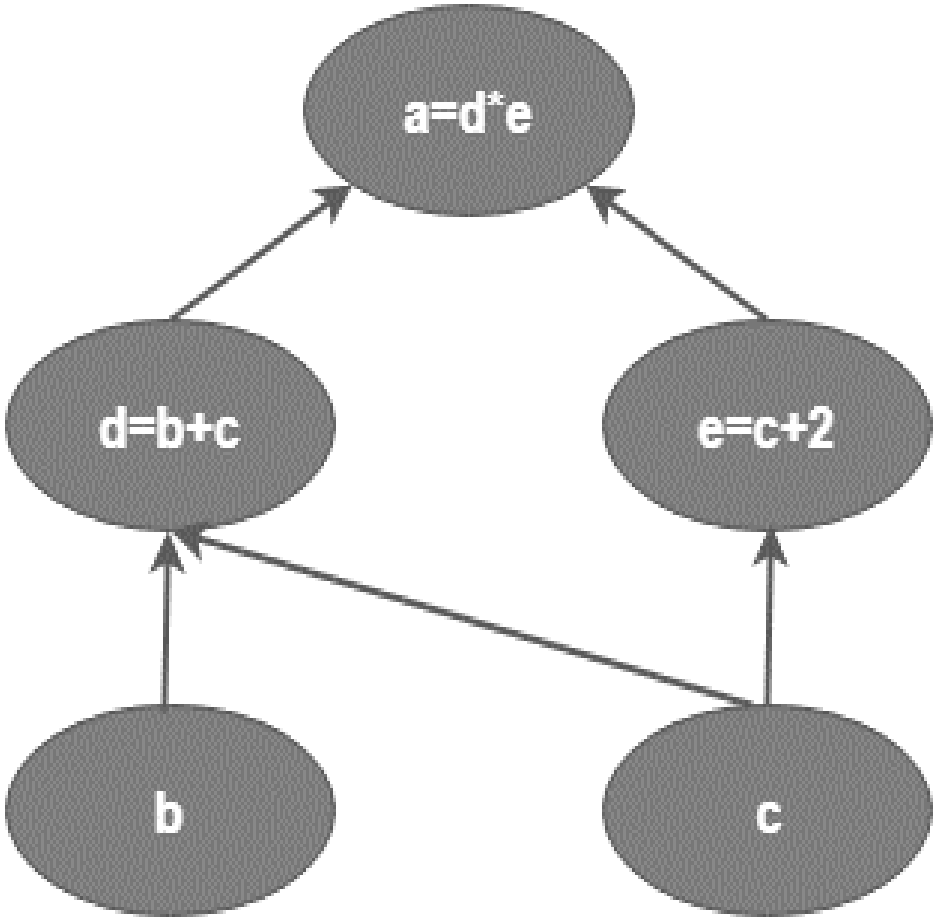
❖ We can break the functions into components given below:

d=b+c

e=c+2

a=d*e

❖ Now, we can represent these operations graphically below:



Session

- A session can execute the operation from the graph. To feed the graph with the value of a tensor, we need to open a session. Inside a session, we must run an operator to create an output.

Why is TensorFlow popular?

- TensorFlow is the better library for all because it is accessible to everyone. TensorFlow library integrates different API to create a scale deep learning architecture like CNN (Convolutional Neural Network) or RNN (Recurrent Neural Network).
- **TensorFlow is based on graph computation; it can allow the developer to create the construction of the neural network with Tensorboard. This tool helps debug our program. It runs on CPU (Central Processing Unit) and GPU (Graphical Processing Unit).**

How TensorFlow Works?

- One of the best things about TensorFlow is that it provides a feature that **helps to create structures for our machine learning and deep learning models**. These structures are made of **dataflow graphs**.
- **Dataflow graphs denote the functionalities that you want to implement.**
- TensorFlow allows developers to create dataflow graphs that describe how data moves through a graph or a series of processing nodes.
- Each node in the graph represents a mathematical operation and each connection or edge between nodes is a multidimensional data array or tensor.
- TensorFlow provides all of this for the programmer through the Python language, which is easy to learn and work with and provides convenient ways to express high-level abstractions.

Advantages

Main advantages of TensorFlow that make it a popular choice for machine learning and deep learning:

- **Highly Scalable:** Works on CPUs, GPUs, and TPUs (Tensor Processing Units) , from mobile to distributed systems.
- **Open Source:** Free to use with strong support from Google and the developer community.
- **Easy to Use with Keras:** High-level API for quick and simple model building.
- **Flexible Architecture:** Supports both eager execution and graph mode.
- **Cross-Platform Deployment:** Models can run on web, mobile, cloud, and edge devices.
- **TensorBoard Support:** Powerful visualization tool for tracking training and performance.
- **Automatic Differentiation:** Calculates gradients automatically for training.
- **Model Saving & Reloading:** Supports saving and loading models easily in various formats.

How TensorFlow Works – Step by Step

1. Define a Computational Graph:

- ❖ You start by building a model using TensorFlow's APIs (like Keras).
- ❖ You define layers, activation functions, loss, and optimizer.
- ❖ Keras is the high-level API of the TensorFlow platform. It provides an approachable, highly-productive interface for solving machine learning (ML) problems, with a focus on modern deep learning.
- ❖ Keras covers every step of the machine learning workflow, from data processing to hyperparameter tuning to deployment.
- ❖ Keras is a platform that simplifies the complexities associated with deep neural networks.

Example

```
1. import tensorflow as tf
2. model = tf.keras.models.Sequential([
3.     tf.keras.layers.Dense(128, activation='relu'),
4.     tf.keras.layers.Dense(10)
5. ])
```

- ❖ Creates a sequential model, where layers are stacked one after another in order.This is useful for simple feedforward (linear) models.
- ❖ Adds a fully connected (dense) layer with:128 neurons (units) ReLU activation function (rectified linear unit), which introduces non-linearity. It learns 128 features from the input data.
- ❖ Adds another dense layer with 10 neurons and no activation function specified.
- ❖ Typically used as the output layer, especially in classification tasks with 10 classes (e.g., MNIST digits 0–9).
- ❖ The output will be raw scores (logits).

2. Tensors – The Core Data

- Tensor = multidimensional array (like a NumPy array).
- All data that flows through the graph (inputs, weights, outputs) are tensors.

3. Sessions & Eager Execution

- Earlier versions used Sessions to run graphs.
- Now, TensorFlow uses Eager Execution by default: operations are executed immediately, like normal Python code.

4. Compile the Model

- You specify:
  - Loss function (e.g., MSE, cross-entropy)
  - Optimizer (e.g., Adam, SGD)
  - Metrics to evaluate.

Example:

1. **model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])**

❖ Compiles the model with the **Adam optimizer** (which adapts learning rates), uses sparse categorical crossentropy as the loss function (suitable for multi-class classification with integer-labeled targets), and monitors accuracy during training to evaluate how often the model's predictions are correct.

5. Train the Model

❖ You feed in training data and labels. TensorFlow:

- ❖ Feeds data through the model
- ❖ Calculates loss
- ❖ Uses backpropagation to adjust weights

1. **model.fit(x_train, y_train, epochs=5)**

❖ Trains the model using the training data x_train (inputs) and y_train (labels) for 5 complete passes (epochs) through the dataset, allowing the model to learn patterns from the data.

6. Evaluate the Model

❖ Test the model on new (unseen) data:

1. **model.evaluate(x_test, y_test)**

7. Make Predictions

❖ Use the model to predict outputs for new inputs:

1. **predictions = model.predict(new_data)**

Handwritten Digit Recognition using Neural Networks with TensorFlow (MNIST Dataset)

Understanding MNIST

- The MNIST dataset, or Modified National Institute of Standards and Technology dataset, is a widely-used collection of handwritten digits designed for training and testing image classification systems. It includes 60,000 training images and 10,000 testing images, all of which are grayscale and 28×28 pixels in size.

1. import tensorflow as tf

- We import the TensorFlow library and give it a shortcut name tf, so we can use its tools easily.

 # Load MNIST data (handwritten digits)

1. (x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

- ❖ MNIST is a dataset of 70,000 handwritten digit images (0–9).
- ❖ x_train: Training images (60,000 samples).
- ❖ y_train: Labels for training images.
- ❖ x_test: Test images (10,000 samples).
- ❖ y_test: Labels for test images.

 # Normalize input data (0–255 → 0–1)

1. **x_train = x_train / 255.0**
2. **x_test = x_test / 255.0**

❖ Images have pixel values from 0 to 255. We divide by 255 to scale all values between 0 and 1. This helps the model learn faster and more accurately.

 # Flatten 28x28 images into 784-length vectors

❖ **x_train = x_train.reshape(-1, 28 * 28)**

❖ **x_test = x_test.reshape(-1, 28 * 28)**

❖ Each image is 28x28 pixels = 784 pixels. We reshape it into a 1D array of 784 values because the **Dense layer** takes flat input.

```
# Build the model
model = tf.keras.models.Sequential([
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(10)  # 10 classes (digits 0–9)
])
```

- This is a simple neural network with:
- First Layer: 128 neurons, ReLU activation (detects patterns).
- Second Layer: 10 neurons (one for each digit from 0 to 9).
- No activation in the output layer because we'll use softmax later while predicting.

 # Compile the model

1. **model.compile(optimizer='adam',**
   loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
   metrics=['accuracy'])

- ❖ Optimizer: 'adam' is an algorithm that helps the model learn.
- ❖ Loss Function: SparseCategoricalCrossentropy compares predictions with true labels.
- ❖ Metrics: We track 'accuracy' to see how well the model performs.
- ❖ Sparse Categorical Crossentropy internally converts these integer labels into one-hot encoded format before calculating the loss. This approach can save memory and computational resources, especially when dealing with datasets containing a large number of classes.

 # Train the model

1. **model.fit(x_train, y_train, epochs=5)**

❖ We train the model using training data for 5 rounds (epochs). The model learns patterns between the images and the correct digit labels.

 # Evaluate the model on test data

1. **model.evaluate(x_test, y_test)**

❖ We test how well the model performs on unseen test data (x_test, y_test). It returns loss and accuracy.

 # Example: Predict using new data (here we take first 5 test samples as demo)

1. **new_data = x_test[:5] # Use any data of shape (n_samples, 784)**
2. **predictions = model.predict(new_data)**

❖ We take the first 5 test images and use model.predict() to get predictions.

 # Display predictions (logits)

1. **print("Raw predictions (logits):")**
2. **print(predictions)**

❖ tf.argmax finds the index (0–9) of the highest score for each prediction. That index is the predicted digit.

 # If you want to see predicted digit classes:

1. **predicted_classes = tf.argmax(predictions, axis=1)**
2. **print("Predicted classes:", predicted_classes.numpy())**

```
Epoch 1/5
1875/1875 [==============================] - 11s 5ms/step - loss: 0.2551 - accuracy: 0.9278
Epoch 2/5
1875/1875 [==============================] - 11s 6ms/step - loss: 0.1119 - accuracy: 0.9671
Epoch 3/5
1875/1875 [==============================] - 11s 6ms/step - loss: 0.0769 - accuracy: 0.9771
Epoch 4/5
1875/1875 [==============================] - 10s 5ms/step - loss: 0.0584 - accuracy: 0.9823
Epoch 5/5
1875/1875 [==============================] - 9s 5ms/step - loss: 0.0453 - accuracy: 0.9857
313/313 [==============================] - 1s 3ms/step - loss: 0.0730 - accuracy: 0.9776
1/1 [==============================] - 0s 130ms/step
Raw predictions (logits):
[[ -2.0148253   -7.4765687   -1.4432983    1.3067949  -11.9206085
   -7.574869   -13.464501    10.944326    -4.4088855    0.364129  ]
 [ -4.4630265    2.5496747   13.344672    -2.3034916  -17.844488
   -2.0982864   -7.7756987  -19.357864     0.43647572 -11.429084  ]
 [ -7.138501     5.970764    -3.4610229   -5.7297444   -2.0162404
   -6.2601514   -6.94954     -1.8291457   -2.3635564   -7.2305937 ]
 [ 12.630065    -9.365913     1.1298823   -5.802517    -4.1931453
   -6.2192764   -0.05513725  -2.8910732   -7.52963     -2.0440516 ]
 [ -2.705968    -5.3162336   -2.7666497   -7.926244    11.539877
   -9.398073    -5.1294017   -1.0413995   -3.6964223    4.028975  ]]
Predicted classes: [7 2 1 0 4]
```

Computational Graph

- A computational graph is a fundamental concept in many deep learning frameworks, including TensorFlow and PyTorch.
- It is a graphical representation of a mathematical computation that consists of nodes and edges.
- Let's investigate deeper into what a computational graph is and why it's important:

Components of a Computational Graph:

- **Nodes (Vertices):** Nodes represent operations or computations. These operations can be mathematical functions, such as addition or matrix multiplication, or more complex operations like convolution in a neural network. Each node takes one or more inputs, performs a specific operation, and produces an output.
- **Edges (Directed Edges):** Edges represent the flow of data between nodes. They indicate the input-output relationships between operations. Data, in the form of tensors (multi-dimensional arrays), flows through these edges as it moves from one node to another.

- **Tensors:** Tensors are the data that flows through the computational graph. They can be scalars (single values), vectors (1D arrays), matrices (2D arrays), or higher-dimensional arrays. Tensors are the inputs, outputs, and intermediate values generated by the operations in the graph.
- **A Partial differential equation (PDE) is the primary type of differential equation, which involves partial derivative with the unknown function of several independent variables. Regarding partial differential equations, we are focusing on creating new graphs.**

Linear Regression in TensorFlow

- Linear regression is a fundamental machine learning algorithm used for modeling the relationship between a dependent variable (target) and one or more independent variables (features) by fitting a linear equation.
- TensorFlow is a popular deep learning framework that can be used to implement linear regression models as well. Here's a step-by-step guide on how to perform linear regression in TensorFlow:

Import Libraries:

- First, you need to import the necessary libraries, including TensorFlow and any other libraries you might need for data manipulation and visualization.

1. ***import tensorflow as tf***
2. ***import numpy as np***
3. ***import matplotlib.pyplot as plt***

Prepare Data:

❖ You'll need to have your data in the form of NumPy arrays or TensorFlow tensors. Let's create some synthetic data for this example:

```
# Generate some synthetic data
np.random.seed(0)
X = np.random.rand(100, 1)
y = 2 * X + 1 + 0.1 * np.random.randn(100, 1)
```

❖ This code generates 100 random input values and computes corresponding output values using the linear equation y = 2X + 1 with added noise to simulate real-world data.

Build the Model:

❖ In TensorFlow, you can create a linear regression model using the tf.keras.Sequential API. This is a simple single-layer model with one neuron.

```
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(1,)),
    tf.keras.layers.Dense(1)  # One neuron for linear regression
])
```

Compile the Model:

❖ Before training, you need to compile the model by specifying the loss function and optimizer. For linear regression, Mean Squared Error (MSE) is a common choice for the loss function.

1. ***model.compile(loss='mean_squared_error', optimizer='sgd')***

Train the Model:

❖ Now, you can train the model using your data.

1. ***history = model.fit(X, y, epochs=100, verbose=1)***

Evaluate the Model:

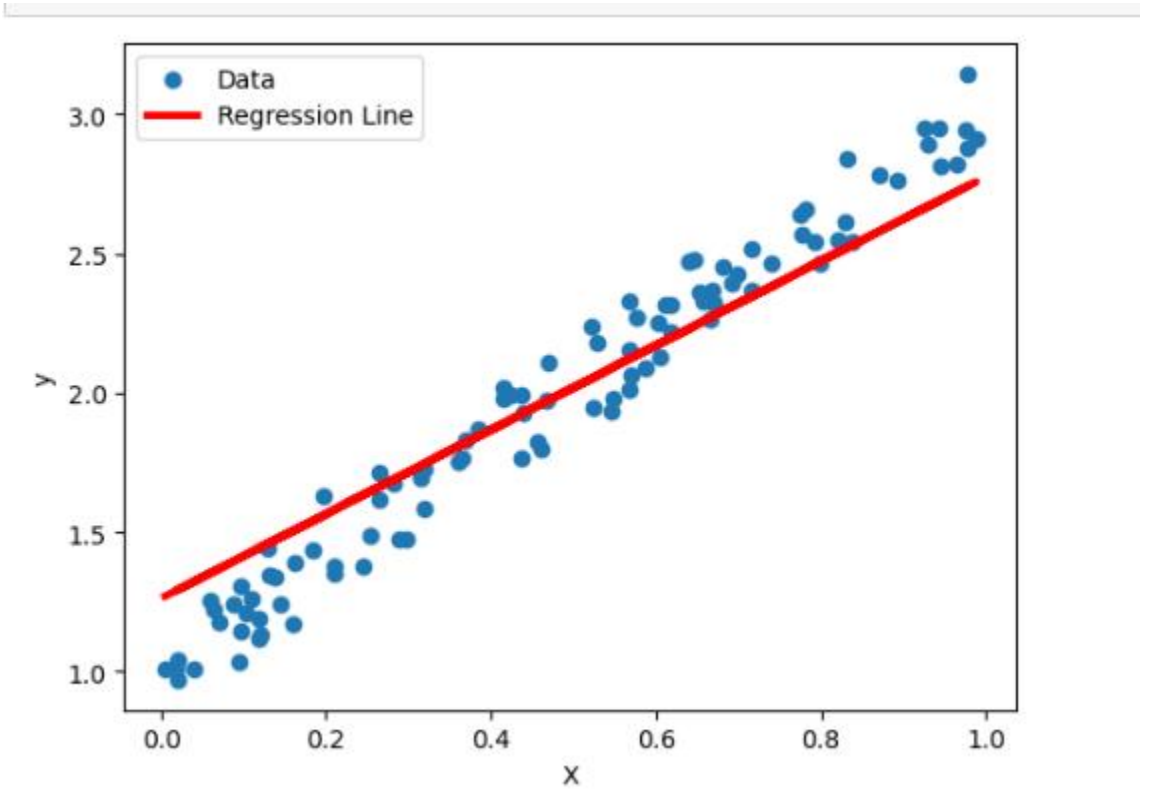
❖ After training, you can evaluate the model's performance and see how well it fits the data.

1. ***# Get the model's predictions***
2. ***predictions = model.predict(X)***
3. ***# Calculate the MSE on the training data***
4. ***mse = np.mean((predictions - y)**2)***
5. ***print(f"Mean Squared Error: {mse}")***

Visualize the Results:

❖ You can visualize the training process and the model's predictions.

1. ***plt.scatter(X, y, label='Data')***
2. ***plt.plot(X, predictions, color='red', linewidth=3, label='Regression Line')***
3. ***plt.legend()***
4. ***plt.xlabel('X')***
5. ***plt.ylabel('y')***
6. ***plt.show()***



Gradient Descent

- Gradient Descent is a fundamental optimization algorithm used in machine learning and deep learning to minimize the loss or cost function of a model during training.
- TensorFlow, as a popular deep learning framework, provides a wide range of tools and functions to implement gradient descent optimization.
- Gradient descent is an optimization algorithm commonly used to train machine learning models, including neural networks.
- It works by iteratively updating the parameters of a model in the direction of the negative gradient of the loss function.
- The loss function measures how well the model performs on the training data, and the goal of gradient descent is to minimize the loss function.

Gradient-based learning is the backbone of many deep learning algorithms. This approach involves iteratively adjusting model parameters to minimize the loss function, which measures the difference between the actual and predicted outputs.

Gradient Descent

- TensorFlow is a popular open-source software library for machine learning and deep learning.
- It provides a number of features that make it easy to implement gradient descent optimization, including:
- **Automatic differentiation:** TensorFlow can automatically compute the gradients of any function, which simplifies the process of implementing gradient descent.
- **Built-in optimizers:** TensorFlow provides a number of built-in optimizers, such as the Adam optimizer, which can be used to implement different variants of gradient descent.
- **Distributed training:** TensorFlow can be used to train models on distributed systems, which can significantly speed up the training process.

To implement gradient descent optimization in TensorFlow, you can follow these steps:

- **Define your model:** Define your model using TensorFlow tensors and operations.
- **Define your loss function:** Define a loss function that measures how well your model performs on the training data.
- **Choose an optimizer:** Choose a TensorFlow optimizer to implement gradient descent.
- **Define your training loop:** In your training loop, perform the following steps:
  - Compute the gradients of the loss function with respect to the model parameters.
  - **Apply the gradients to the model parameters using the chosen optimizer**.
  - Evaluate the model on the training data and update the loss function.
- Repeat steps 4 and 5 until the model converges.

What is a Perceptron

- A perceptron is one of the simplest artificial neural network architectures, and it serves as the fundamental building block of more complex neural networks.
- It was developed by Frank Rosenblatt in the late 1950s.
- **A perceptron is primarily used for binary classification tasks, where it takes a set of input values, applies weights to those inputs, sums them up, and then passes the result through an activation function to produce an output. This output is typically either 0 or 1, making it suitable for binary decisions.**

What is Perceptron and how it has been developed?

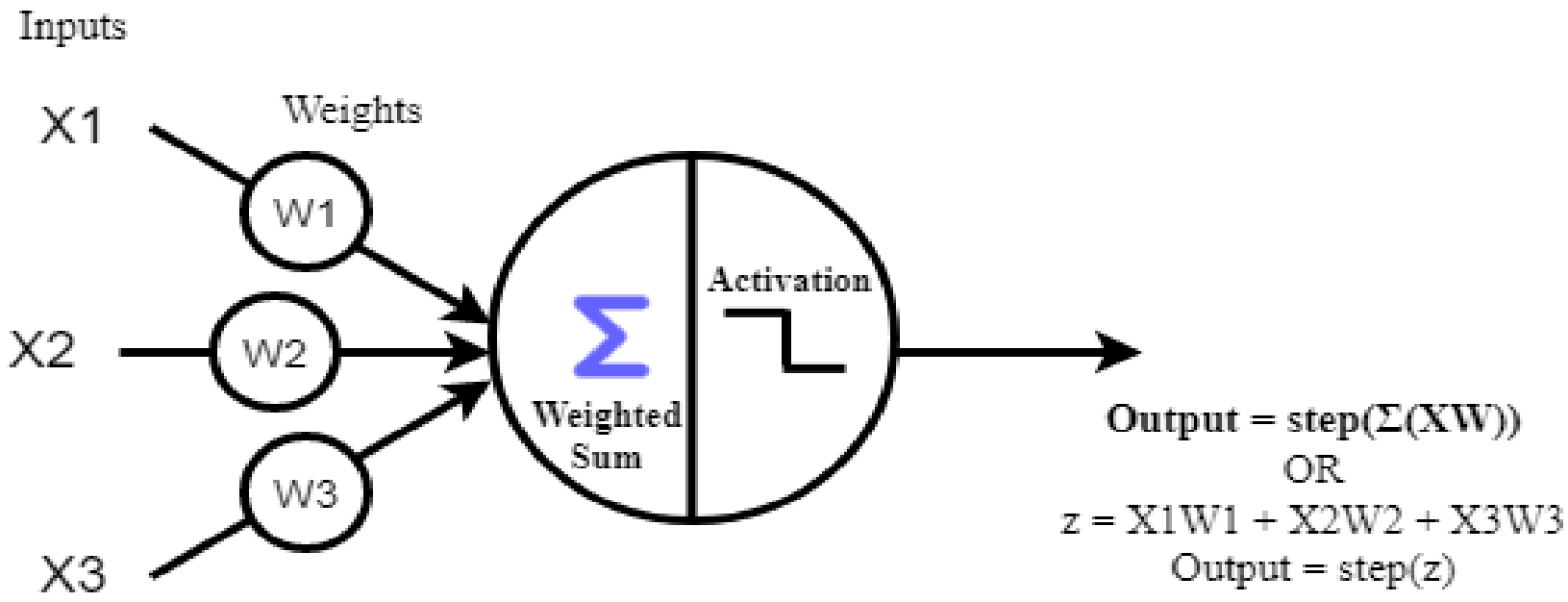
- The word Perceptron is first ever coined in **1943 by Warren McCulloch and Walter Pitts. Lately, Frank Rosenblatt,** an American psychologist, and computer scientist built the first Perceptron machine when doing research at **Cornell Aeronautical Laboratory** for performing image recognition.
- During this time the Perceptron machine become popular among the AI community and is considered the fundamental part of building intelligent systems.

- **The perceptron algorithm is based on the concept of a single neuron in the human brain, a single neuron in the human brain is doing a very simple thing, just receiving some inputs and if the inputs are high, it activates and send the signal to the next neuron.**
- The perceptron was designed to **mimic (copycat) this process**, with the input data serving as the input to the neuron and the weights representing the strength of the connections between the input neurons and the output neuron.

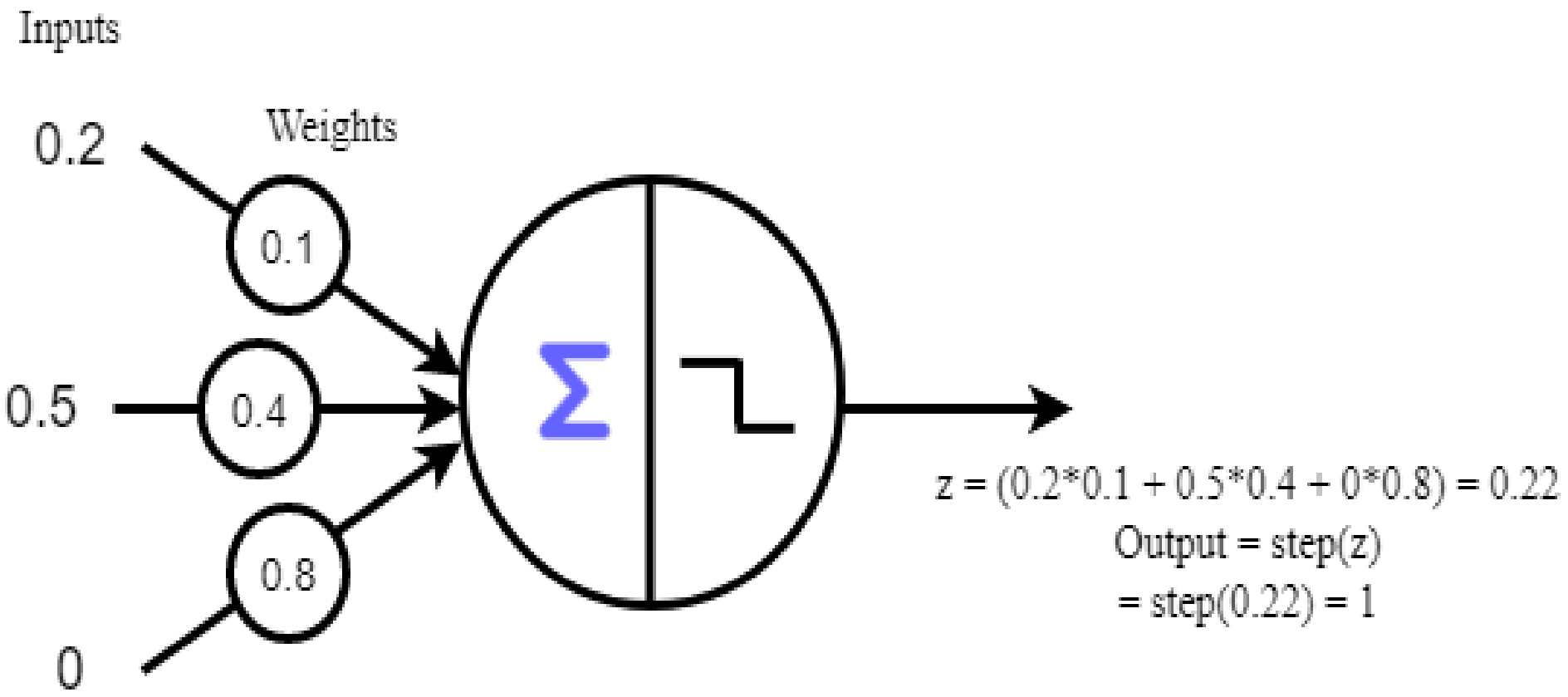
Perceptron is a single-layer neural network linear or a Machine Learning algorithm used for supervised learning of various binary classifiers. It works as an artificial neuron to perform computations by learning elements and processing them for detecting the business intelligence and capabilities of the input data.

The Perceptron Algorithm, How does it work?

- The Perceptron is a type of linear classifier (binary classifier), which means it can be used to classify data that is linearly separable.
- A Perceptron is somehow similar to Logistic Regression at first glance, but it's different. While Logistic Regression predicts probabilities of a data point falling in a particular class, Perceptron will only tell whether the data point is in a particular class or not, Just like saying "Yes" or "No". Here is the diagrammatic representation of the perceptron algorithm.



- A Perceptron is a kind of single Artificial Neuron which is also known as a Threshold Processing Unit / Threshold Logic Unit (TLU).
- As you can see in the above diagram, the Perceptron contains some input links X1, X2, and, X3.
- Each input has its own corresponding weights W1, W2, and W3. These weights are basically the hearts of Perceptrons which determine the strength of each input signal to it.
- The Perceptron or Threshold Logic Unit (TLU) computes the weighted sum of the inputs ($z = X1W1 + X2W2 + X3W3 ..... + XnWn$) and then these weighted sums of inputs are passed through an activation function also known as the step function.
- These activation functions will determine whether the Perceptron needs to be activated or not.
- Let's see an example to understand this,


Inputs
Weights
0.2
0.1
0.5
0.4
0.8
0
Σ
z = (0.2*0.1 + 0.5*0.4 + 0*0.8) = 0.22
Output = step(z)
= step(0.22) = 1

Heaviside function

❖ The Heaviside activation function will return 0 when the weighted sum of inputs is less than zero and return 1 if it is greater than or equal to 0.]

$$\text{heaviside(z)} = \left\{ \begin{array}{c} 0\ if\ z < 0 \\ 1\ if\ z \geq 0 \end{array} \right\}$$

Sign function

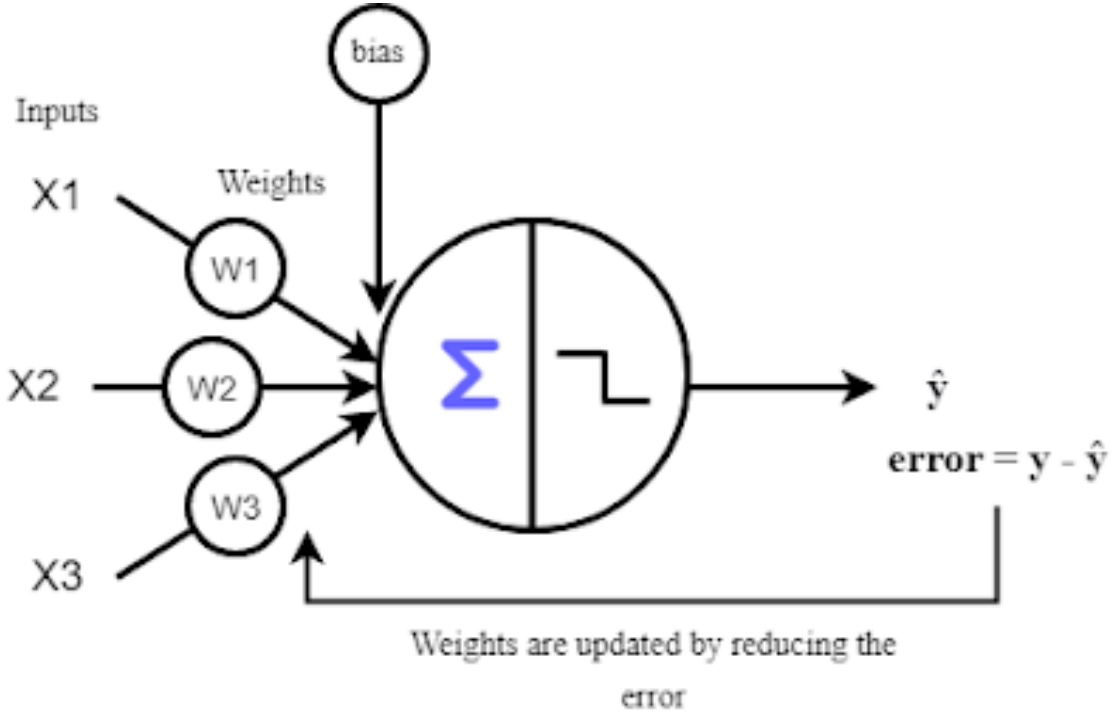
❖ The Sign function will return 0 if the weighted sum of inputs is 0 and return +1 and -1 when the weighted sum of inputs is greater and lesser than 0 respectively.

$$\text{sgn(z)} = \left\{ \begin{array}{c} -1\ if\ z < 0 \\ 0\ if\ z = 0 \\ +1\ if\ z > 0 \end{array} \right\}$$

Learning in Perceptron

- The whole point of training the perceptron is to adjust the weights of each input according to the training data.
- **For training the perceptron to find the best weights, Frank Rosenblatt proposed an algorithm that is largely inspired by Hebb's rule.**
- **Hebb's rule state that when a biological neuron triggers another neuron, the connection between these two neurons grows stronger.**
- **There is an awesome phrase for this property of neurons, ie, Neurons that fire together wire together.**
- Inspired by Hebb's rule, the Perceptron can be trained by finding the error made by the algorithm and finding the weights that reduce this error as much as possible.
- This is the main goal of training a machine learning algorithm in general.

- The algorithm will first make a prediction based on the training data given, then this prediction is compared with the actual desired output.
- The error is calculated for each of the training instances by comparing the prediction and actual output, finally, this error is reduced during the training process and the weights corresponding to the correct predictions are adjusted accordingly.



y = Actual values
ŷ = Prediction

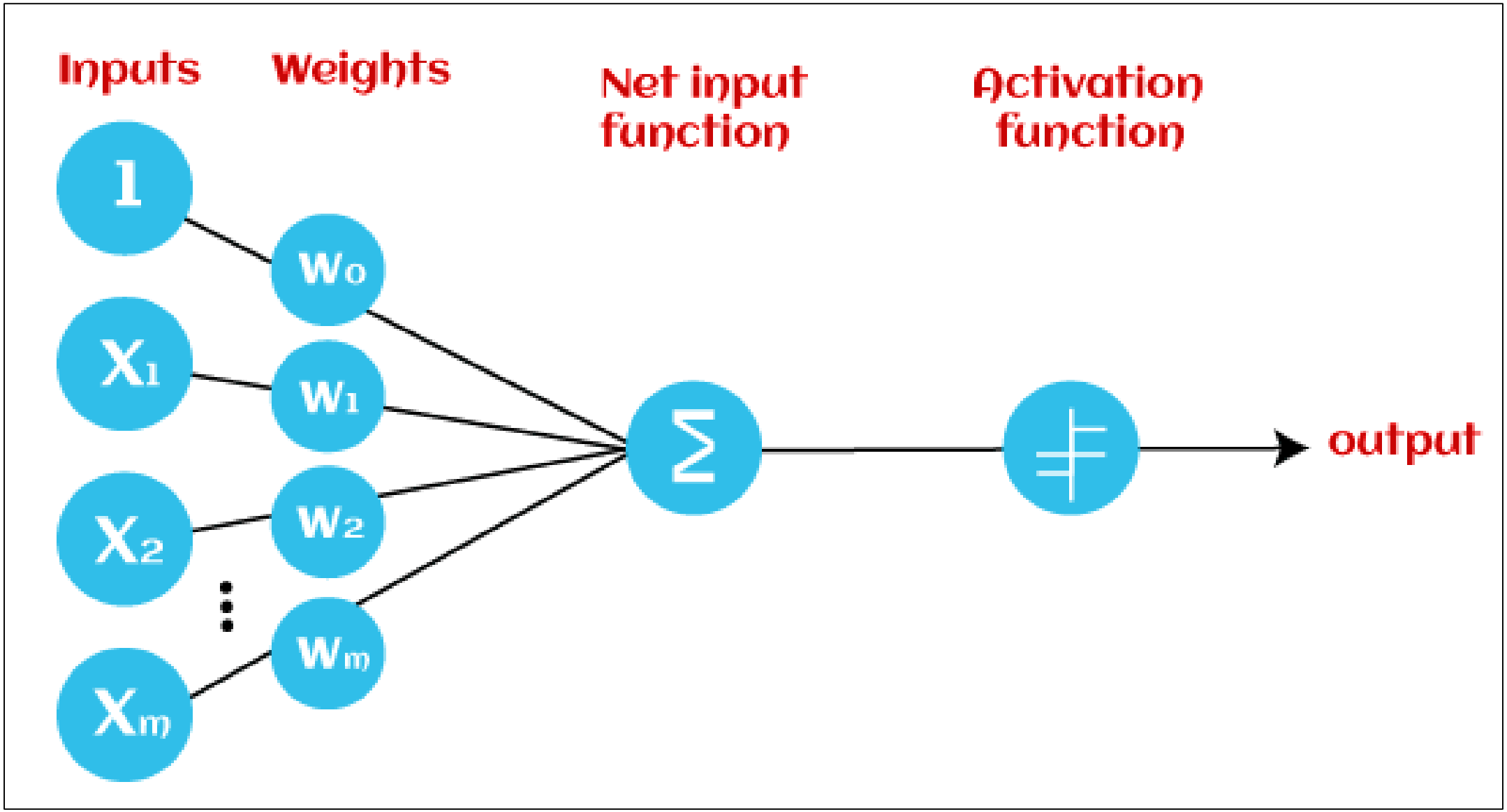
- The learning(training) rule can be written as

$$w_{i,j}(next) = w_{i,j} + \eta(y_i - \hat{y})x_i$$

  - Where wi, j is the weights between ith input neuron and jth output neuron
  - **η(eta) is the learning rate**
  - y is the target output or the actual values
  - ŷ is the predicted value of the output neuron
  - xi is the input value of the current training instance
- The algorithm is being trained by updating the weights until getting the optimal results.
- The learning rate(η) is required to determine the length of each step taken for finding optimal weights. Sometimes, it is better to have a smaller learning rate to get a minimum error, but sometimes a large learning rate is also preferred.

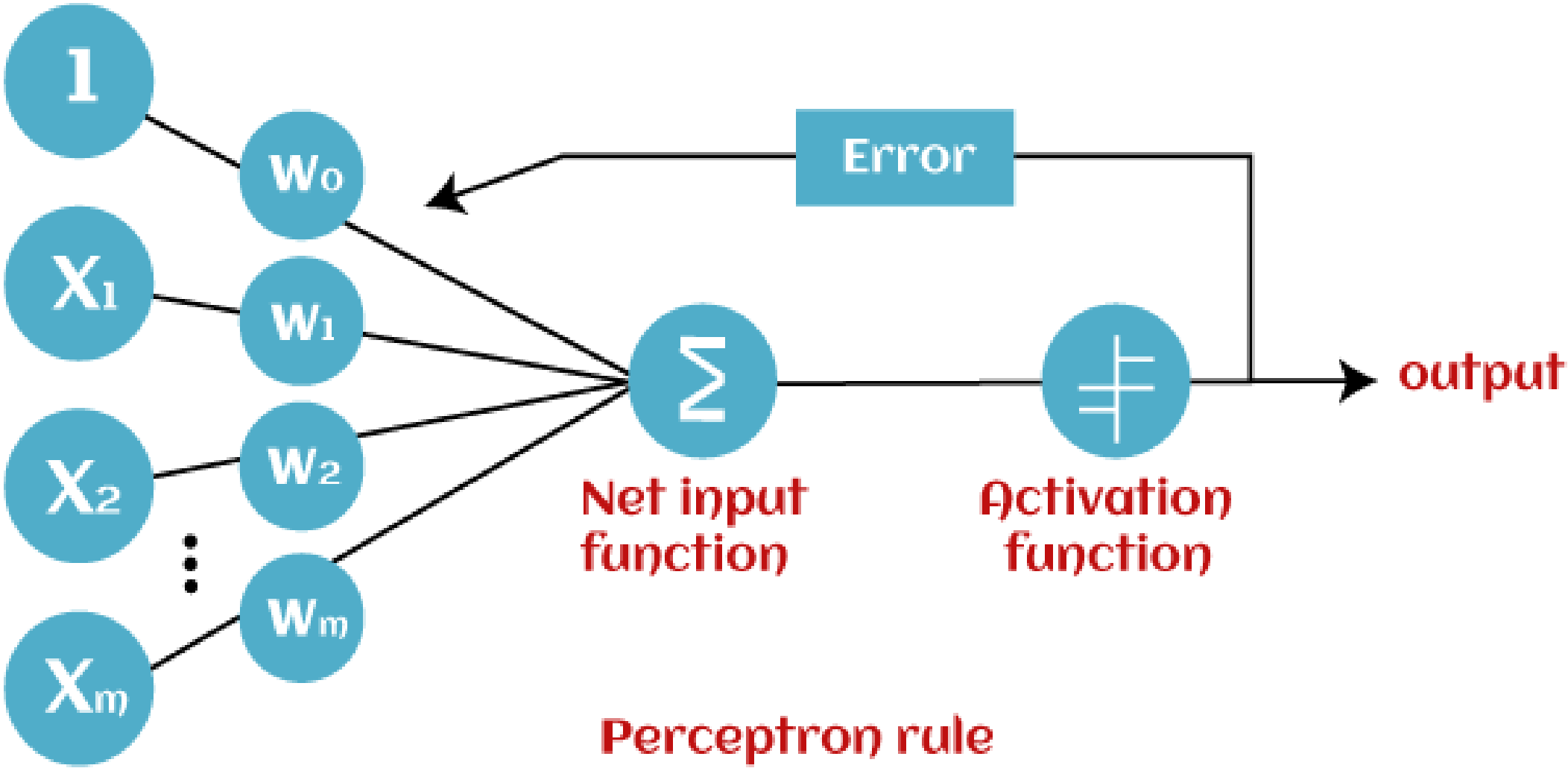
❖ Basic Components of Perceptron

- ❖ Mr. Frank Rosenblatt invented the perceptron model as a binary classifier which contains three main components. The basic components of a perceptron are:
- ❖ **Input nodes:** Also known as the input layer, these nodes accept the initial data for processing. Each input node has a real numerical value.
- ❖ **Weights:** Represent the strength of the connection between units.
- ❖ **Bias:** A part of the perceptron.
- ❖ **Weighted sum:** A function within the neuron that produces an output based on the numerical value of the inputs and the weights
- ❖ **Activation function:** Applies to the input to produce a binary output. Activation functions are important for building multilayer perceptrons and can add non-linearity to the structure. These are the final and important components that help to determine whether the neuron will fire or not.


Inputs
Weights
Net input function
Activation function
1
X1
X2
Xm
W0
W1
W2
Wm
Σ
output

How does Perceptron work? What are the working Steps?

- Perceptron is considered as a single-layer neural network that consists of four main parameters named **input values (Input nodes), weights and Bias, net sum, and an activation function.**
- The perceptron model begins with the multiplication of all input values and their weights, then adds these values together to create the weighted sum.
- Then this weighted sum is applied to the activation function 'f' to obtain the desired output.
- This activation function is also known as the step function and is represented by 'f'.
- This step function or Activation function plays a vital role in ensuring that output is mapped between required values (0,1) or (-1,1).
- It is important to note that the weight of input is indicative of the strength of a node. Similarly, an input's bias value gives the ability to shift the activation function curve up or down.



Perceptron rule

- Perceptron model works in two important steps as follows:

Step-1:

- In the first step first, multiply all input values with corresponding weight values and then add them to determine the weighted sum. Mathematically, we can calculate the weighted sum as follows:
- $\sum wi*xi = x1*w1 + x2*w2 + \ldots wn*xn$
- Add a special term called bias 'b' to this weighted sum to improve the model's performance.
- $\sum wi*xi + b$

Step-2

- In the second step, an activation function is applied with the above-mentioned weighted sum, which gives us output either in binary form or a continuous value as follows:
- $Y = f(\sum wi*xi + b)$

Perceptron Algorithm for XOR Logic Gate

- The basic perceptron algorithm is not capable of learning the XOR logic gate because XOR is a non-linearly separable problem.
- In other words, it's not possible to draw a single straight line (or hyperplane) to separate the inputs that produce a 1 (true) output from those that produce a 0 (false) output in XOR.
- The XOR truth table looks like this:

Input 1	Input 2	Output
0	0	0
0	1	1
1	0	1
1	1	0

- There's no way to draw a single straight line to separate the 1s from the 0s in a 2D space (representing the inputs), which is what the perceptron algorithm does.
- The perceptron algorithm works for problems that are linearly separable, where a single line can separate the classes.
- However, you can still solve the XOR problem by using a more complex neural network, such as a multi-layer perceptron (MLP) or a feed forward neural network with hidden layers.
- These networks introduce non-linearities through activation functions in the hidden layers, allowing them to learn and model more complex relationships in the data.
- Here's a simple Python example of how you can use an MLP to solve the XOR problem using the Keras library (part of TensorFlow):

- The XOR gate can be represented using the perceptron algorithm by combining two perceptrons: OR and NAND:
  - OR: 2*1+2*2–1
  - NAND -x1-x2+2
- The XOR gate can also be represented as a combination of an OR gate (x1 + x2), a NAND gate (-x1-x2+1), and an AND gate (x1+x2–1.5).
- Perceptrons are machine learning algorithms that use supervised learning of binary classifiers.
- In a perceptron, the weight coefficient is automatically learned by multiplying weights with input features.
- The activation function then uses a step rule to determine **if the function is greater than zero.** If the sum of all input values is greater than the threshold value, the output signal will be shown. Otherwise, there will be no output

```
import tensorflow as tf
from tensorflow import keras
# Define the XOR dataset
X = [[0, 0], [0, 1], [1, 0], [1, 1]]
y = [0, 1, 1, 0]
# Create a sequential model
model = keras.Sequential([
    keras.layers.Input(shape=(2,)),
    keras.layers.Dense(2, activation='relu'),  # Hidden layer with ReLU activation
    keras.layers.Dense(1, activation='sigmoid')  # Output layer with sigmoid activation
])
# Compile the model
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
# Train the model
model.fit(X, y, epochs=100, verbose=0)
# Evaluate the model
loss, accuracy = model.evaluate(X, y)
print(f'Loss: {loss}, Accuracy: {accuracy}')
```

```
1/1 [==============================] –
0s 312ms/step - loss: 0.6504 - accuracy: 1.0000
Loss: 0.6504468321800232, Accuracy: 1.0
```

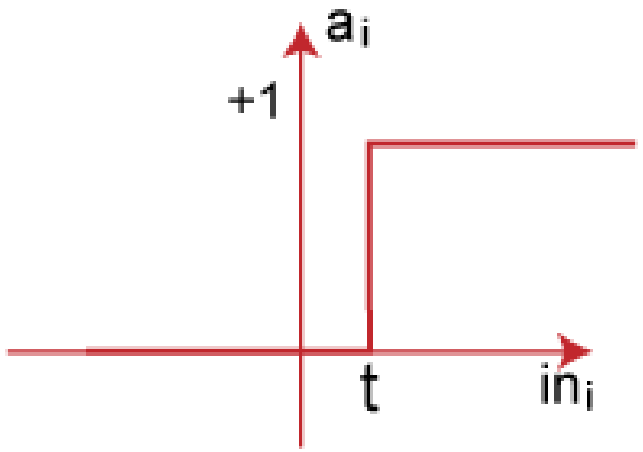
Activation Functions: Sigmoid, ReLU, Hyperbolic Fns, Softmax

- Perceptron is Machine Learning algorithm for supervised learning of various binary classification tasks. Further, Perceptron is also understood as an Artificial Neuron or neural network unit that helps to detect certain input data computations in business intelligence.
- Perceptron model is also treated as one of the best and simplest types of Artificial Neural networks. However, it is a supervised learning algorithm of binary classifiers.
- Hence, we can consider it as a single-layer neural network with four main parameters, i.e., **input values, weights and Bias, net sum, and an activation function.**
- **In Machine Learning, binary classifiers are defined as the function that helps in deciding whether input data can be represented as vectors of numbers and belongs to some specific class.**
- Binary classifiers can be considered as linear classifiers. In simple words, we can understand it as a classification algorithm that can predict linear predictor function in terms of weight and feature vectors.

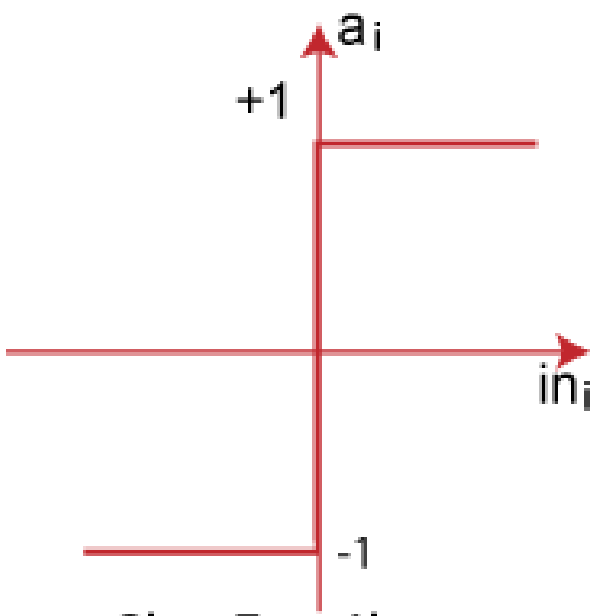
Types of Activation functions:

- ❖ Sign function
- ❖ Step function, and
- ❖ Sigmoid function

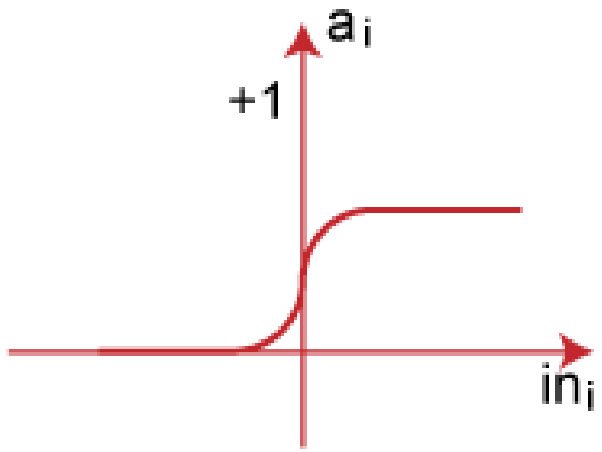
- ❖ The data scientist uses the activation function to take a subjective decision based on various problem statements and forms the desired outputs. Activation function may differ (e.g., Sign, Step, and Sigmoid) in perceptron models by checking whether the learning process is slow or has vanishing or exploding gradients.



Step Function



Sign Function



Sigmoid Function

Activation Functions: Sigmoid

- An activation function is a crucial component of a perceptron, as it determines the output of the perceptron based on the weighted sum of its inputs. One common activation function used in perceptrons is the sigmoid function.
- The sigmoid activation function, also known as the logistic function, is a widely used activation function in neural networks. It has a characteristic S-shaped curve and is defined mathematically as follows:

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

In this equation:

- σ(z) is the output of the sigmoid function for a given input
- e is the base of the natural logarithm, approximately equal to 2.71828.

Here are some key properties and characteristics of the sigmoid activation function:

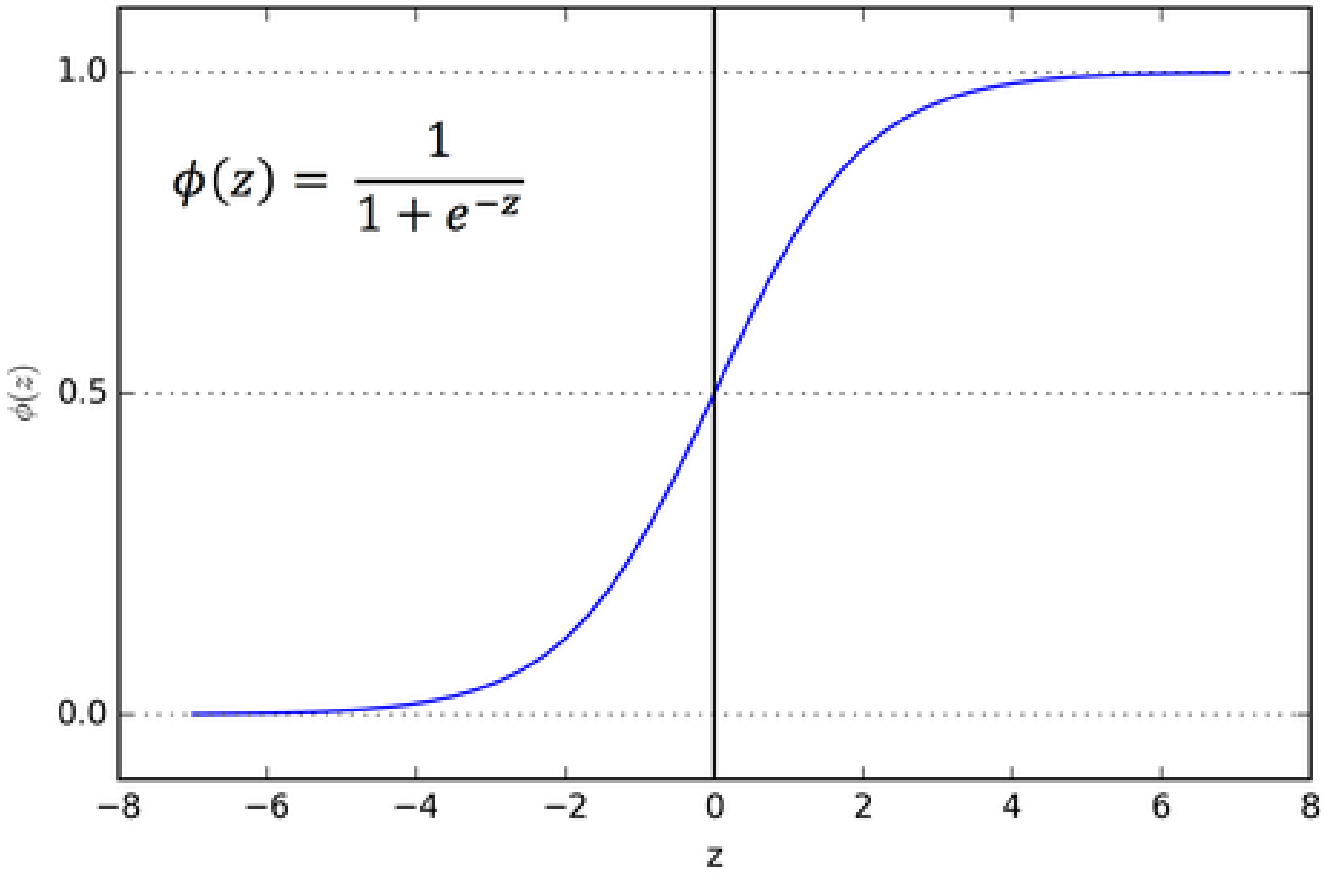
- **Output Range:** The sigmoid function always produces output values in the range (0, 1). This makes it suitable for binary classification problems because it can be interpreted as a probability. Values close to 0 represent one class, and values close to 1 represent the other class.
- **Smoothness:** The sigmoid function is smooth and continuous, which makes it differentiable everywhere. This differentiability is crucial for training neural networks using gradient-based optimization algorithms like gradient descent.
- **Non-Linearity:** The sigmoid function introduces non-linearity into the model. Without non-linearity, stacking multiple perceptrons together would still result in a linear model, limiting its ability to represent complex relationships in data.
- **Vanishing Gradient:** One drawback of the sigmoid function is that it suffers from the vanishing gradient problem. When the input to the sigmoid is very large (positive or negative), the gradient of the sigmoid becomes very small. This can slow down the learning process or make it difficult for the model to converge, especially in deep networks.

- The sigmoid activation function is a mathematical function that transforms a real number into a value between 0 and 1.
- **Used in neural networks as an activation function, where it applies a non-linear function after each matrix multiplication**.
- **This allows the network to learn functional relationships beyond linear ones**
- The sigmoid function is defined as:

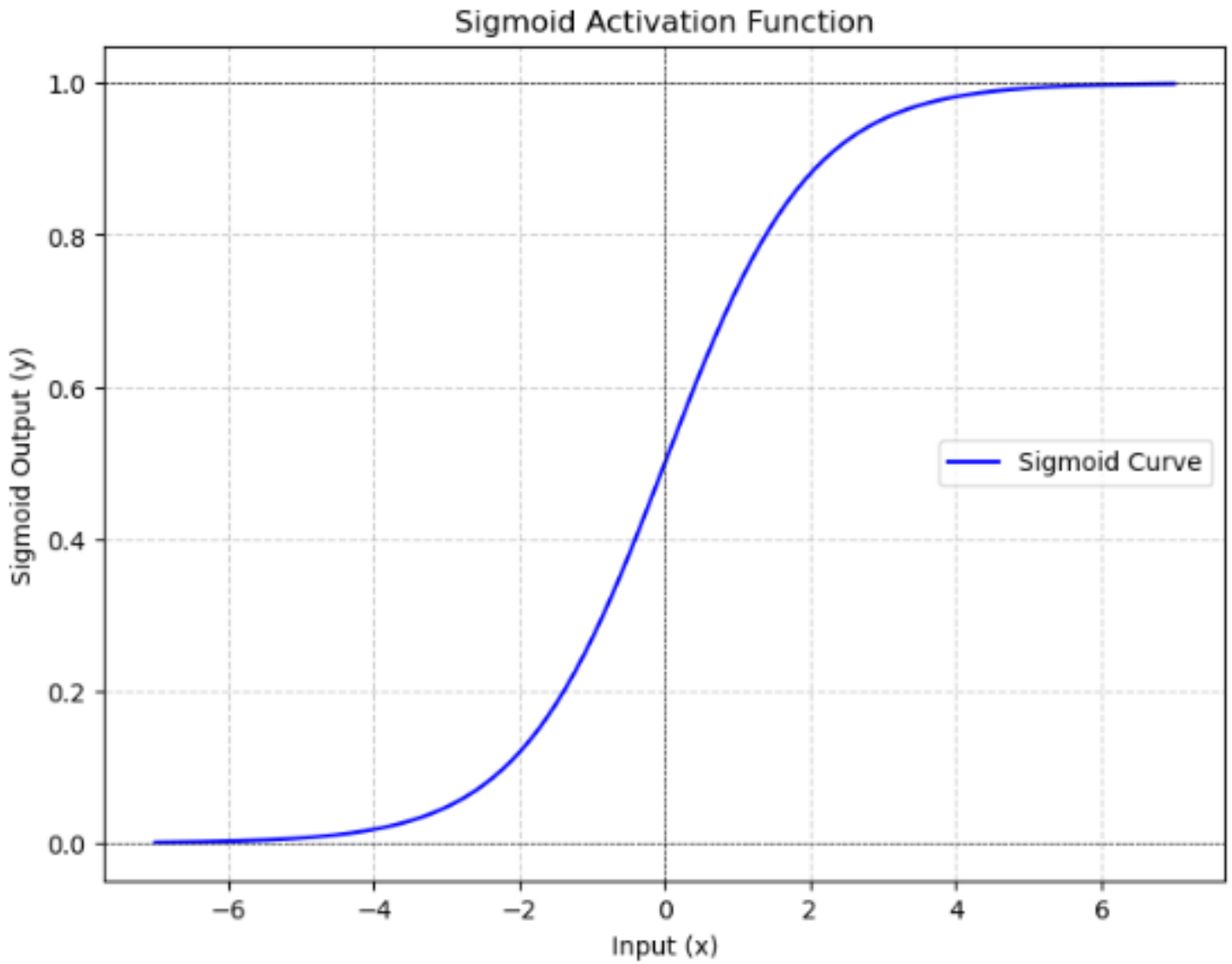
$$f(x) = \frac{1}{1 + e^{-x}}$$

- Here are some properties of the sigmoid function:
- **Input:** The function takes any real value as input
- **Output:** The output value is close to 0 when the input is small (negative), and close to 1 when the input is large (positive)
- **Domain:** The domain is (-∞, +∞)
- **Range:** The range is (0, +1)
- **Continuity:** The function is continuous everywhere
- **Differentiability:** The function is differentiable everywhere in its domain

- The sigmoid function is used in multilabel classification, where the output layer has one node for each mutually inclusive class.
- It can also be used to convert raw outputs from the final layer of a deep learning model into final scores between 0 and 1.



```
import numpy as np
import matplotlib.pyplot as plt
# Define the sigmoid activation function
def sigmoid(z):
    return 1 / (1 + np.exp(-z))
# Generate values for the x-axis
x = np.linspace(-7, 7, 200)  # Create 200 equally spaced values from -7 to 7
# Calculate sigmoid values for the x-axis values
y = sigmoid(x)
# Plot the sigmoid curve
plt.figure(figsize=(8, 6))
plt.plot(x, y, label='Sigmoid Curve', color='blue')
plt.xlabel('Input (x)')
plt.ylabel('Sigmoid Output (y)')
plt.title('Sigmoid Activation Function')
plt.axhline(y=0, color='black', linestyle='--', linewidth=0.5)
plt.axhline(y=1, color='black', linestyle='--', linewidth=0.5)
plt.axvline(x=0, color='black', linestyle='--', linewidth=0.5)
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend()
plt.show()
```


Sigmoid Activation Function
Sigmoid Output (y)
1.0
0.8
0.6
0.4
0.2
0.0
−6
−4
−2
0
2
4
6
Input (x)
Sigmoid Curve

Perceptron with Activation Functions: ReLU

- A perceptron with the Rectified Linear Unit (ReLU) activation function is a type of artificial neural network unit that uses the ReLU function as its activation function. ReLU is one of the most commonly used activation functions in modern neural networks, including deep learning architectures.
- Here's an explanation of the perceptron with ReLU activation:
- **Input Layer:** Similar to a standard perceptron, a perceptron with ReLU activation takes one or more input values. These inputs are typically real-valued features or the outputs of other neurons in the network.
- **Weights and Bias:** Each input is associated with a weight, which represents the strength of the connection between the input and the neuron. Additionally, there is a bias term, which allows the model to shift the decision boundary.
- **Weighted Sum:** The perceptron calculates the weighted sum of the inputs by multiplying each input by its corresponding weight and then summing them up, just like in a standard perceptron:

$$z = \sum_{i=1}^{n} (x_i \cdot w_i) + b$$

Where:

- z is the weighted sum,
- Xi are the input values,
- Wi are the corresponding weights,
- n is the number of inputs, and
- b is the bias term.

- The ReLU function works by applying a simple mathematical operation to the input value:
- **If the input value is greater than or equal to zero: The output is equal to the input value.**
- **If the input value is negative: The output is zero**
- ReLU introduces non-linearity to a deep learning model and **solves the vanishing gradients issue**. **It returns 0 if the input is negative, but for any positive input, it returns that value back. The function can be written as**

$$f(x)=max(0,x).$$

The ReLU activation function is used to introduce nonlinearity in a neural network, helping mitigate the vanishing gradient problem during Deep Learning model training and enabling neural networks to learn more complex relationships in data

- **ReLU has several advantages, including:**
  - **Training complex models:** ReLU can train complex models and neural networks with constant values.
  - Efficiency: ReLU runs faster than sigmoid functions and has a lower run time
  - Sparse activation - ReLU outputs a zero value for negative inputs, which leads to sparse activations. This means that only a subset of neurons are activated at any given time, making the network more efficient.
  - ReLU helps to mitigate the vanishing gradient problem, allowing neural networks to learn more effectively from backpropagation.
  - Convergence: ReLU accelerates the convergence of stochastic gradient descent compared to the sigmoid and tanh functions.
  - Training: ReLU trains quickly and efficiently

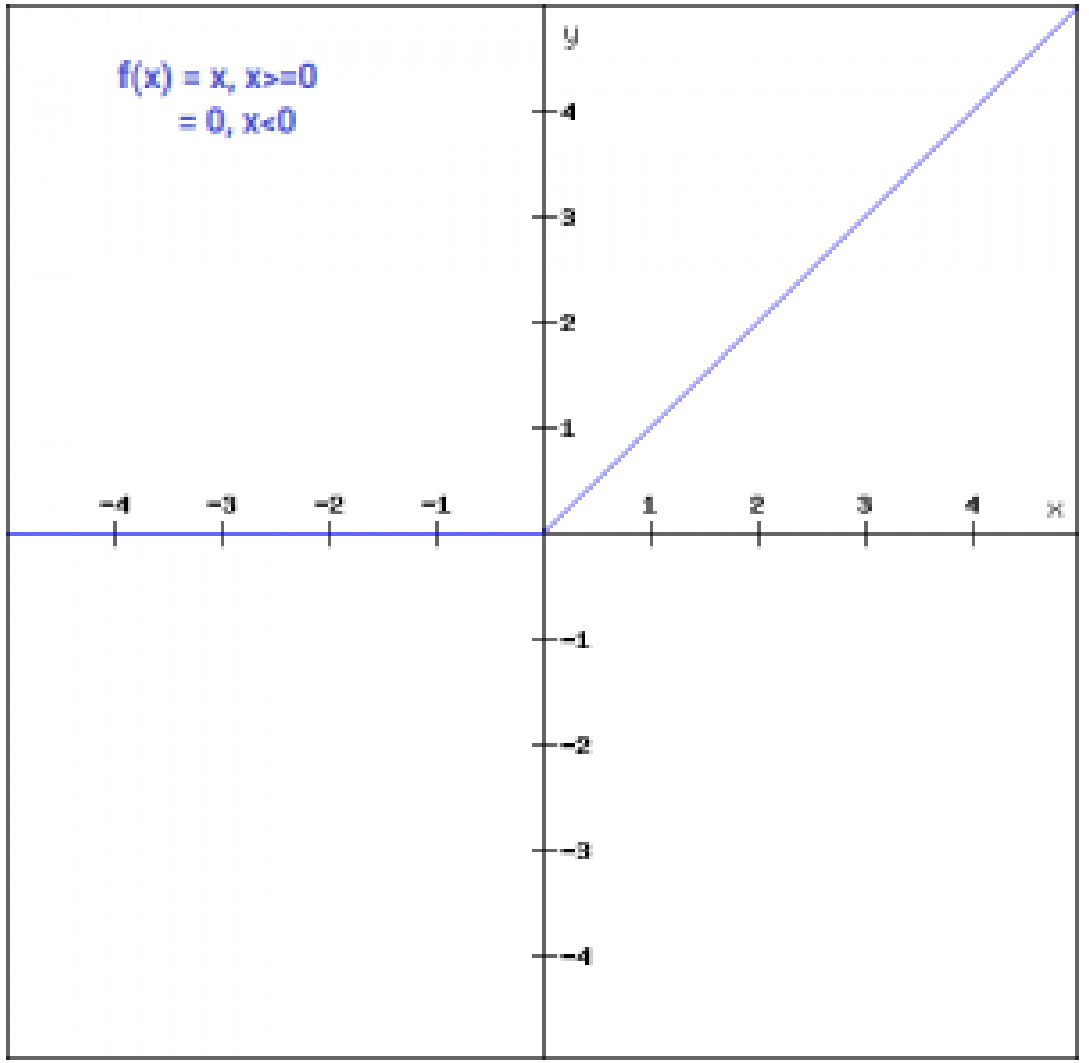
- **Disadvantages of the ReLU Activation Function**
- The main issue with ReLU is that all the negative values become zero immediately, which decreases the ability of the model to fit or train from the data properly.
- ReLU also has some problems, such as **exploding gradient**. Exploding gradient occurs when large error gradients accumulate and result in very large updates to neural network model weights during training.

The **exploding gradient problem** is when the gradients in a deep neural network become too large during backpropagation, leading to unstable updates and preventing effective learning. **It happens when the gradients of the network's loss with respect to the parameters (weights) become too large.**

Convergence in machine learning refers to the point where the model's predictions stop improving, or the error rate becomes constant. If a model is not converging, it means that it's not reaching a point of stability where it can make accurate predictions.

- ReLU is common because it's simple to implement and effective at overcoming the limitations of other previously popular activation functions, such as Sigmoid
- The main advantage of using the ReLU function over other activation functions is that **it does not activate all the neurons at the same time.**

- **This means that the neurons will only be deactivated if the output of the linear transformation is less than 0.**
- The plot below will help you understand this better-
- f(x)=max(0,x)



❖ For the negative input values, the result is zero, that means the neuron does not get activated. Since only a certain number of neurons are activated, the ReLU function is far more computationally efficient when compared to the **sigmoid and tanh function**. Here is the python function for ReLU:

```
def relu_function(x):
    if x<0:
        return 0
    else:
        return x
relu_function(7), relu_function(-7)
```

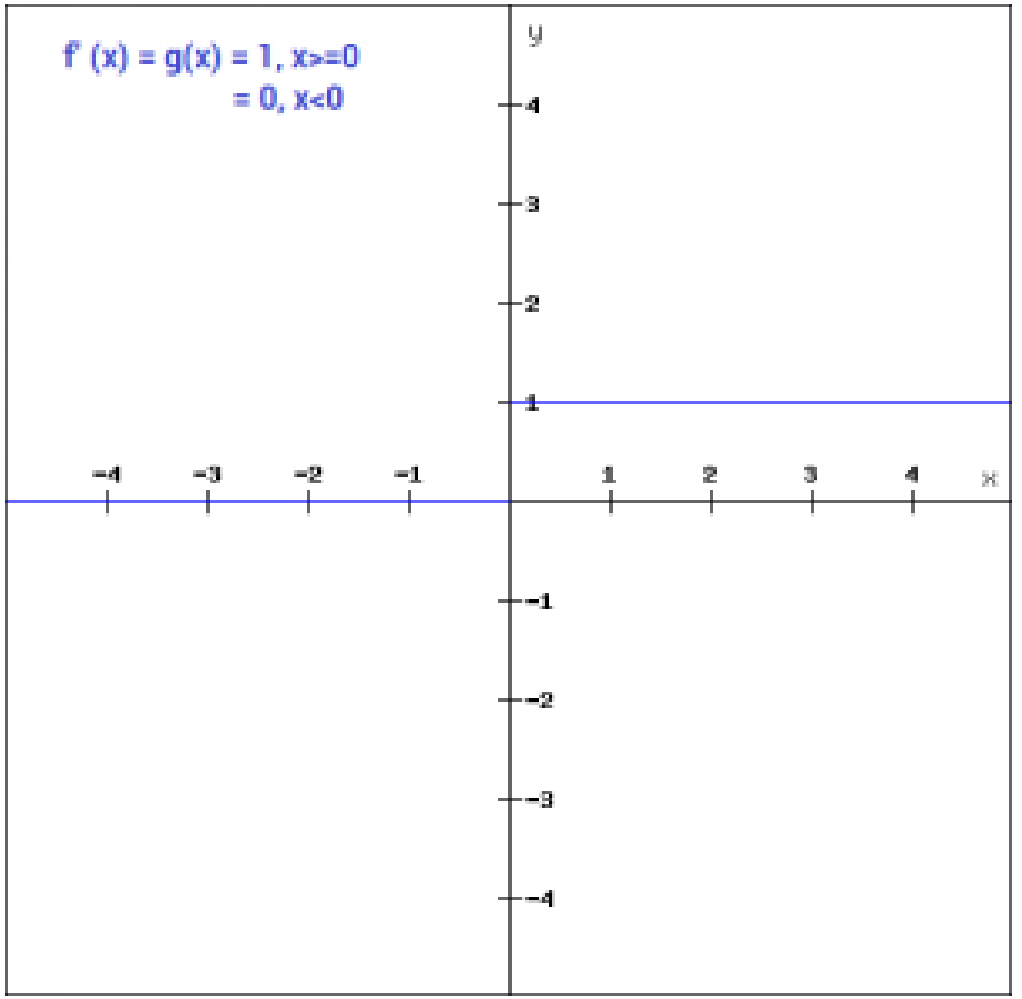
Output:

(7, 0)

Let's look at the gradient of the ReLU function.

f'(x) = 1, x>=0

= 0, x<0



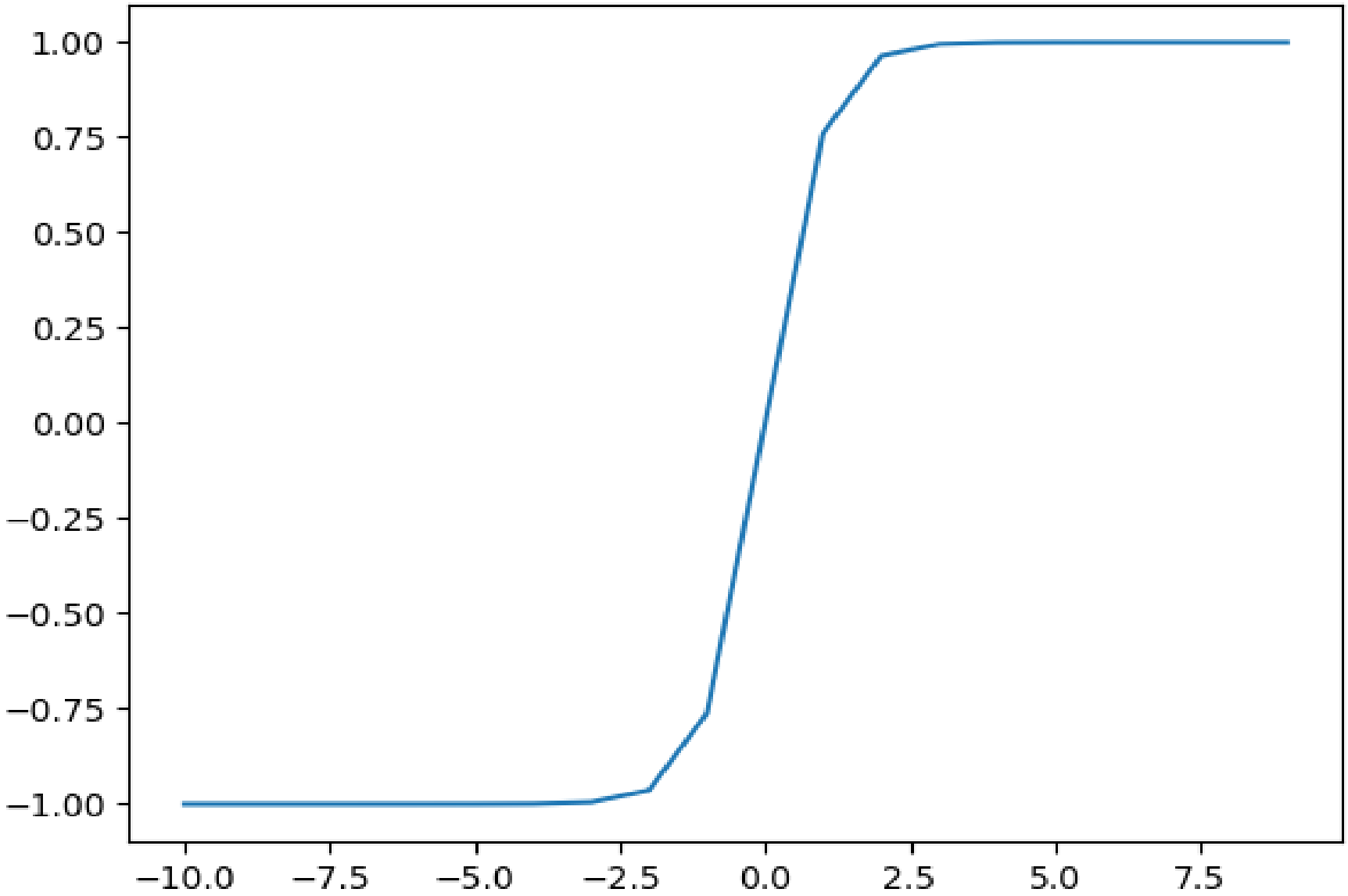
Hyperbolic Tanh Hidden Layer Activation Function

- The hyperbolic tangent activation function is also referred to simply as the Tanh (also "tanh" and "TanH") function.
- It is very similar to the sigmoid activation function and even has the same S-shape.
- **The function takes any real value as input and outputs values in the range -1 to 1.**
- **The larger the input (more positive), the closer the output value will be to 1.0, whereas the smaller the input (more negative), the closer the output will be to -1.0.**
- The Tanh activation function is calculated as follows:

$$Tanh$$

$$f(x) = \frac{(e^{x} - e^{-x})}{(e^{x} + e^{-x})}$$

```
# Example plot for the tanh activation function
	from math import exp
	from matplotlib import pyplot
# tanh activation function
	def tanh(x):
	return (exp(x) - exp(-x)) / (exp(x) + exp(-x))
# define input data
	inputs = [x for x in range(-10, 10)]
# calculate outputs
	outputs = [tanh(x) for x in inputs]
# plot inputs vs outputs
	pyplot.plot(inputs, outputs)
	pyplot.show()
```


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5

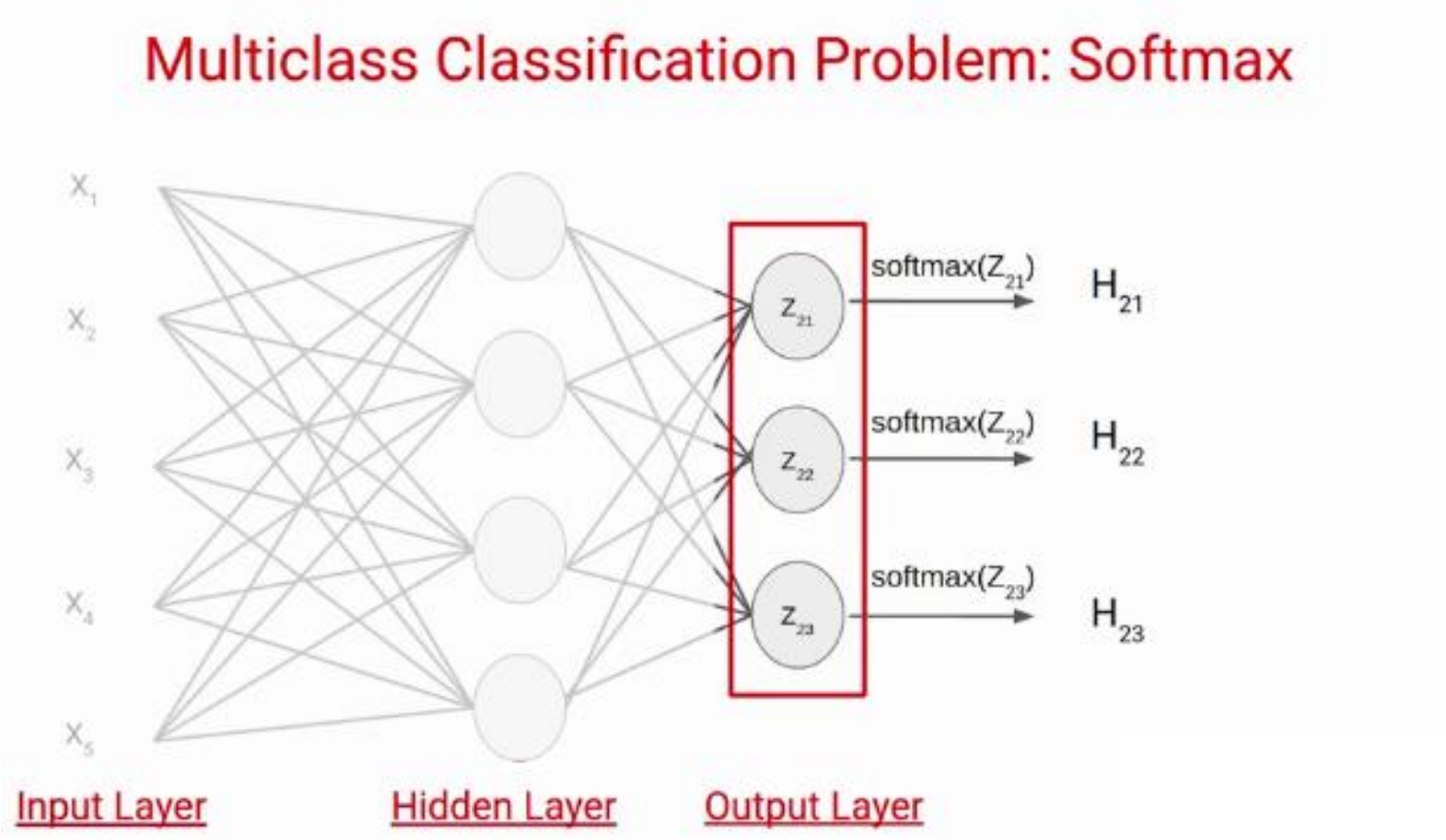
Softmax Activation

- The softmax activation function is a machine learning tool that **converts a vector of real numbers into a probability distribution.**
- Similar to the sigmoid activation function the SoftMax function returns the probability of each class. Here is the equation for the SoftMax activation function.

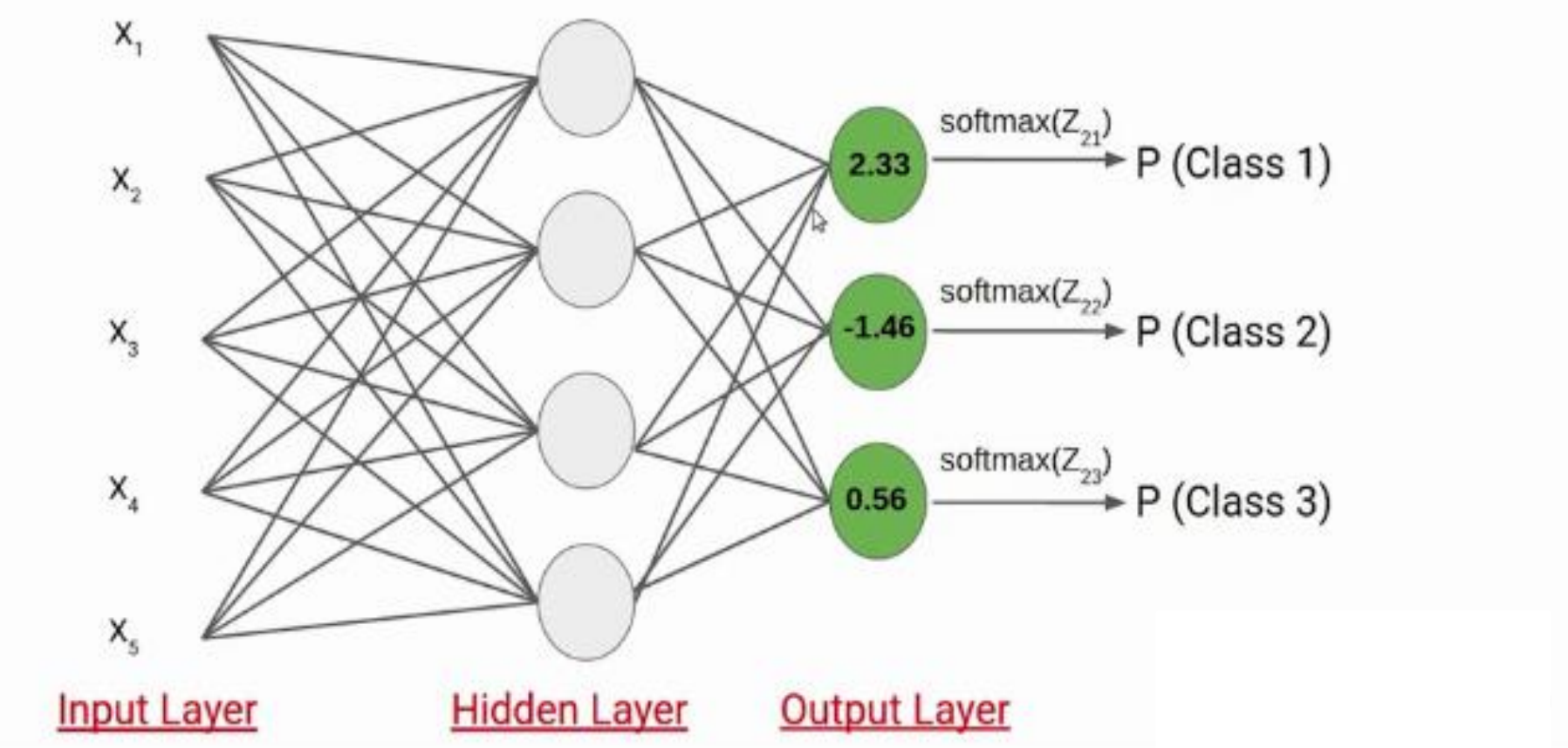
$$softmax(z_i) = \frac{exp(z_i)}{\sum_j exp(z_j)}$$

- Here, the Z represents the values from the neurons of the output layer.
- The exponential acts as the non-linear function.
- Later these values are divided by the sum of exponential values in order to normalize and then convert them into probabilities.

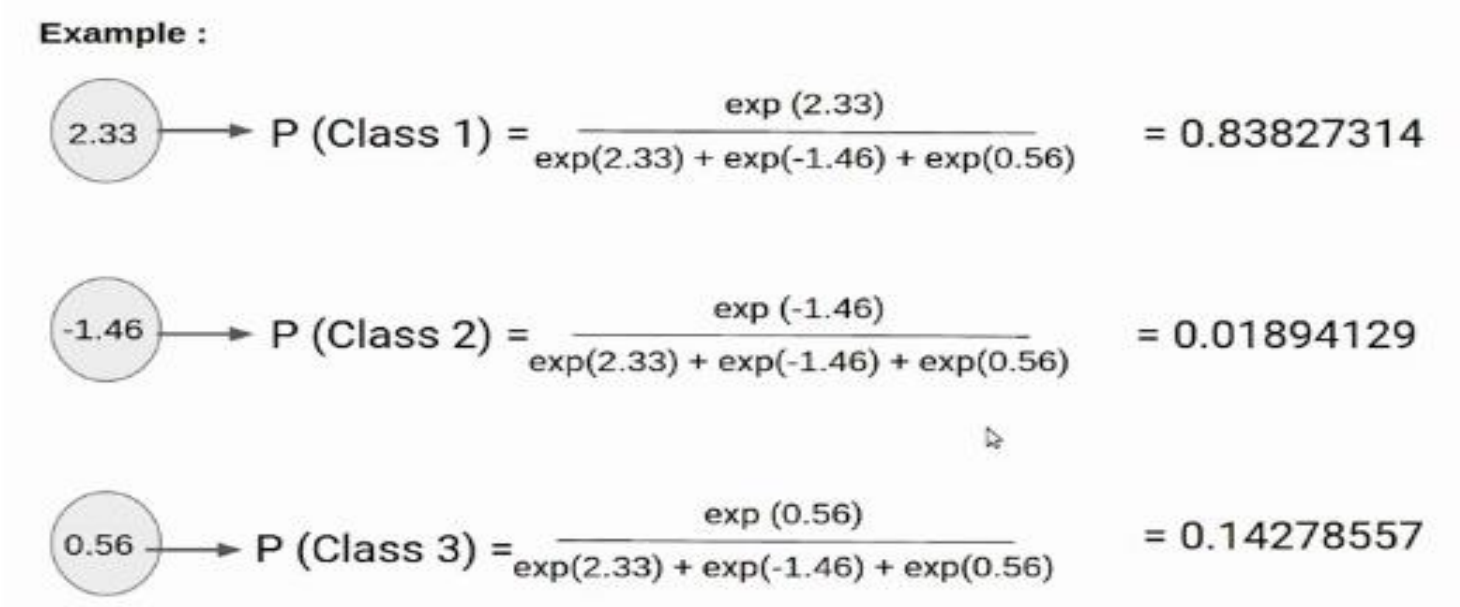
❖ Let’s understand with a simple example how the softmax works, We have the following neural network



❖ Suppose the value of Z_{21}, Z_{22}, Z_{23} comes out to be 2.33, -1.46, and 0.56 respectively. Now the SoftMax activation function is applied to each of these neurons and the following values are generated.



❖ These are the probability values that a data point belonging to the respective classes. Note that, the sum of the probabilities, in this case, is equal to 1.



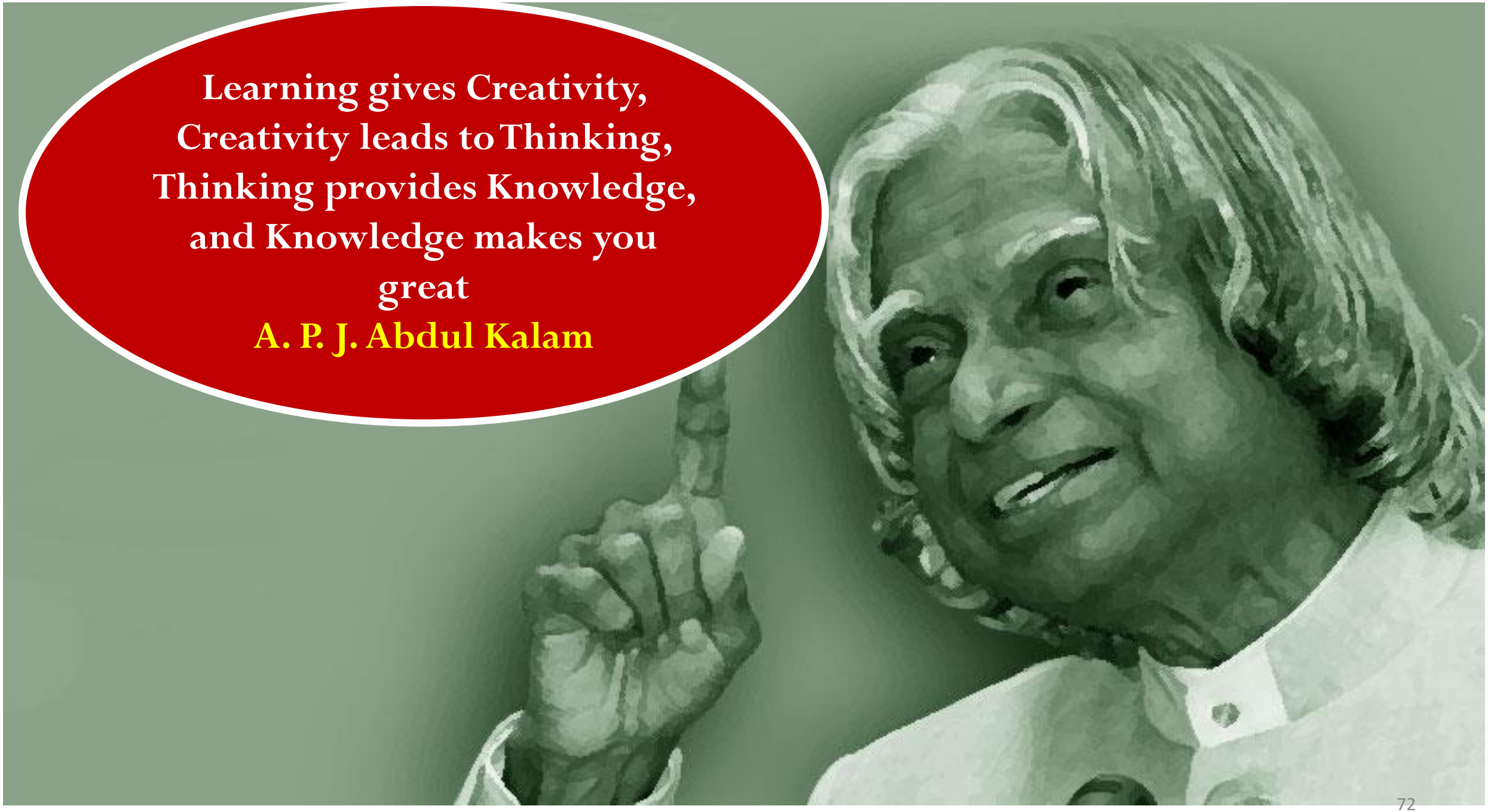
❖ In this case it clear that the input belongs to class 1. So if the probability of any of these classes is changed, the probability value of the first class would also change.

What is the softmax function?

- The softmax function is a mathematical function that converts a vector of real numbers into a probability distribution.
- It exponentiates each element, making them positive, and then normalizes them by dividing by the sum of all exponentiated values.
- This ensures that the output probabilities add up to one, making it suitable for multiclass classification tasks.

What is the difference between sigmoid and softmax functions?

- The sigmoid function is used for binary classification, mapping any real value to a range between 0 and 1. It’s suitable for independent predictions.
- The softmax function, on the other hand, converts a vector of real numbers into a probability distribution for multiclass classification tasks, ensuring that the sum of the probabilities is **equal to one**.


Learning gives Creativity,
Creativity leads to Thinking,
Thinking provides Knowledge,
and Knowledge makes you
great
A. P. J. Abdul Kalam


Thank
You!